

Lab 4.1: Explaining Phishing Detectors

Note: this is not a complicated lab. We added example code, which is why there are many pages. The code is only an example: you can write your own. Read Sections 1 to 3 first, then do Parts A, B, C and D in order. Every grey box is one notebook cell example.

1. What is this lab about?

When a security detector says “**this URL is phishing**”, the analyst needs to know *why* before blocking the site or warning the users. In this lab you work on one problem: **is this web address (URL) a phishing site?**

- **Part A: build the detectors** in the three ways you met in Labs 1 to 3: **supervised** (every URL has a label), **semi-supervised** (only 5% have a label, the model also learns from the other 95%) and **self-supervised** (the model first learns from the URLs alone, then uses the 5% labels).
- **Part B: explain them** with the models themselves (glass boxes), SHAP and LIME.
- **Part C: test** how much the detector needs outside lookups, and how stable LIME is.
- **Part D: turn the explanation into rules:** build a small Belief Rule-Based expert system (BRB) from the two most important features, like in the BRB slides and lab 3.

2. What you will be able to do afterwards

- Train a phishing detector in a supervised, semi-supervised and self-supervised way, and compare them fairly.
- Read a decision tree and a logistic regression, and explain a black box with SHAP and LIME.
- Discuss the trade-off between accuracy, readability and the number of labels, and say which features an attacker could fake.
- Build a BRB from the two most important features and check whether LIME and SHAP agree with its rules.

3. The ideas, explained simply

The examples use made up numbers; your own numbers will be different. In this lab, always report macro-F1, recall and FAR.

3.1 Lower lines and the self-supervised game

Lower line. If the semi-supervised model scores 0.93, is that good? You only know after you train the same model on the 5% labels alone. That score is the lower line: the unlabelled data only helped if you beat it.

Self-supervised game. We hide some clues of each URL and a network must guess them back, e.g. `nb_dots = 5`, `nb_hyphens = 4`, `length_url = ?`. To win, it learns how clues go together, without anyone telling it what phishing is.

3.2 Glass box, black box, global, local

A **glass box** (a small tree, a logistic regression) can be read directly. A **black box** (a forest of 300 trees, a neural network) is too big to read, so we explain it from outside with SHAP and LIME. A **global** explanation says what the model looks at in general; a **local** one says why one URL got its score.

3.3 SHAP: “sharing the prize fairly”

SHAP starts from the **base value** (the average score of ordinary URLs) and says how much each feature pushed *this* URL's score up or down. The pushes always add up exactly to the model's score:

Made-up example: one URL scored 0.92 phishing

0.50 (base) + 0.25 (domain_age, very new) + 0.12 (phish_hints) + 0.08 (nb_hyphens) + 0.03 (page_rank) - 0.06 (length_url, short) = **0.92**

A **waterfall** plot shows this for one URL. A **bar** plot shows the average push of each feature, and a **beeswarm** shows one dot per URL (red = high value, right = toward phishing). For forests, `shap.TreeExplainer` is exact and fast; for other models, `shap.Explainer` works too.

3.4 LIME: “draw a straight line nearby”

LIME makes thousands of small changes of one URL, asks the model to score them, and fits a straight line through the answers. It shows each feature as a range: `page_rank <= 1.00 +0.16` means this low `page_rank` pushes the score 0.16 toward phishing. Its **fit score** (0 to 1) says how well the line copies the model: below 0.5, do not trust it much. LIME is random, so another seed can give another answer: it may not be **stable**. You test this in Part C.

3.5 The trade-off

The more accurate model (maybe the forest) is usually harder to read, the easy-to-read model (the tree) may miss more attacks, and fewer labels save money but may cost accuracy. Your job is not to find the “best” model but to measure these things and argue which balance fits a real security team. If a simple model is almost as good, or the unlabelled data does not help, that is an interesting finding: write it down.

3.6 The BRB, one more time

Two **antecedents** (your two top features), each with three **referential values** Low / Medium / High, give $3 \times 3 = 9$ **belief rules**, e.g. IF `page_rank` is Low AND `nb_hyperlinks` is Low THEN risk is {Low 0.00, Medium 0.11, High 0.89}. **Evidential Reasoning** combines the active rules; when they disagree, part of the belief becomes **Unknown**. The **utility** (Low 0, Medium 0.5, High 1) turns the beliefs into one phishing score.

4. Setup and dataset

Open Google Colab (or any tool you prefer), create a new notebook, and install the two explanation libraries in the first cell (pandas, scikit-learn and matplotlib are already there). Then upload the dataset with the **folder icon** on the left.

```
!pip install shap lime
```

Dataset: Web Page Phishing Detection (Hannousse & Yahiouche, 2021): **11,430 URLs**, half phishing and half legitimate, **87 features** each; the label is in the column `status`. Get it from <https://data.mendeley.com/datasets/c2gw7fy2j4/3> or Kaggle (“Web page Phishing Detection Dataset”), as `dataset_phishing.csv`. The features come from the URL text (56), the page content (24) and 7 external lookups (`whois_registered_domain`, `domain_registration_length`, `domain_age`, `web_traffic`, `dns_record`, `google_index`, `page_rank`).

Feature	Plain meaning (you will meet these in your explanations)
<code>google_index</code>	1 if Google has NOT indexed the page (common for phishing), 0 if it has.
<code>page_rank</code>	How well-linked the site is, 0 to 10. Phishing sites usually score low.
<code>nb_hyperlinks</code>	How many links the page has. Phishing pages often have very few.
<code>web_traffic</code>	The site's traffic rank. Small = popular; 0 = no rank found.
<code>domain_age</code>	Age of the domain in days. Phishing domains are often new. -1 means unknown.
<code>phish_hints</code>	How many phishing words (login, signin, admin...) are in the URL.
<code>nb_www</code>	How many times “www” appears in the URL.

5. Part A: Train the detectors

Step A1: Load the data

```
import numpy as np
import pandas as pd

df = pd.read_csv("dataset_phishing.csv")    # the file you downloaded
print(df.shape)                           # should print (11430, 89)
print(df["status"].value_counts())         # 5715 phishing, 5715 legitimate
df.head()
```

What you should see

The shape (11430, 89) (87 features, the url text and the status label) and 5,715 URLs of each class.

Step A2: Prepare and split

The label becomes 1 = phishing, 0 = legitimate, and the raw url text is dropped. We split 60 / 20 / 20: **training** to learn, **validation** to choose settings, **test** only for the final scores. stratify keeps 50% phishing in each part.

```
from sklearn.model_selection import train_test_split

# The label: 1 = phishing, 0 = legitimate
y = (df["status"] == "phishing").astype(int)

# The features: everything except the raw URL text and the label.
# .astype(float) avoids type warnings later when we change values.
X = df.drop(columns=["url", "status"]).astype(float)

# 60% train, 20% validation, 20% test, same phishing share in each part
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.4, stratify=y, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, stratify=y_temp, random_state=42)

print(len(X_train), len(X_val), len(X_test))    # 6858 2286 2286
```

Step A3: A helper to score any model

Macro-F1, recall, ROC-AUC and FAR

```
from sklearn.metrics import (f1_score, recall_score, roc_auc_score,
                             confusion_matrix)

def report(model, X, y):
    """The four numbers we use for every model in this lab (3 decimals)."""
    pred = model.predict(X)
    prob = model.predict_proba(X)[:, 1]
    tn, fp, fn, tp = confusion_matrix(y, pred).ravel()
    scores = {
        "macro_F1": f1_score(y, pred, average="macro"),
        "recall":   recall_score(y, pred),           # share of phishing we caught
        "ROC_AUC":  roc_auc_score(y, prob),
        "FAR":      fp / (fp + tn),                 # share of legit wrongly flagged
    }
    return {k: round(float(v), 3) for k, v in scores.items()}
```

5.1 Supervised: all labels

Step A4: Glass box 1: a small decision tree

We try depth 3 and 4 and keep the better one on the validation set. You read the tree in step B1.

```

from sklearn.tree import DecisionTreeClassifier, export_text

best_score, tree = -1, None
for depth in [3, 4]:
    t = DecisionTreeClassifier(max_depth=depth, random_state=42)
    t.fit(X_train, y_train)
    score = report(t, X_val, y_val)["macro_F1"] # choose on VALIDATION
    print("depth", depth, "validation macro-F1:", round(score, 3))
    if score > best_score:
        best_score, tree = score, t
print("kept depth", tree.get_depth())

```

Step A5: Glass box 2: logistic regression

One weight per feature. We scale the features first so the weights can be compared; the Pipeline fits the scaler on training data only.

```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# The Pipeline fits the scaler on training data only. No leakage.
logreg = Pipeline([
    ("scale", StandardScaler()),
    ("lr", LogisticRegression(max_iter=1000)),
])
logreg.fit(X_train, y_train)
print(report(logreg, X_val, y_val))

```

Step A6: The black box: a Random Forest

300 trees trained on all labels. This is the upper line: the best you can hope for when every URL is labelled.

```

from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
print(report(rf, X_val, y_val))

```

What you should see

The validation scores of each model. If you get a ConvergenceWarning, it is only a warning: you can ignore it or raise max_iter.

5.2 Semi-supervised: 5% labels plus unlabelled URLs

Step A7: Hide 95% of the labels, and train the lower lines

We split the *training* set into 5% labelled and 95% unlabelled URLs, and train a forest and a logistic regression on the 5% only. *y_hidden* is kept only to check the guesses in step A8: never train on it.

```

# Pretend that only 5% of the training URLs have a label
X_lab, X_unlab, y_lab, y_hidden = train_test_split(
    X_train, y_train, train_size=0.05, stratify=y_train, random_state=42)
print(len(X_lab), "labelled,", len(X_unlab), "unlabelled")
# y_hidden = the labels we pretend we do not have. NEVER train on it.

# Two lower lines: the same models, trained on the few labels only
rf_few = RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1)
rf_few.fit(X_lab, y_lab)
print("forest, 5% labels:", report(rf_few, X_val, y_val))

logreg_few = Pipeline([
    ("scale", StandardScaler()),
    ("lr", LogisticRegression(max_iter=1000)),
])

```

```
logreg_few.fit(X_lab, y_lab)
print("logreg, 5% labels:", report(logreg_few, X_val, y_val))
```

What you should see

342 labelled, 6516 unlabelled, then the scores of the two lower lines.

Step A8: Pseudo-labelling

As in Lab 2: the forest guesses the unlabelled URLs, keeps only the guesses it is very sure about (CUTOFF = 0.9; you can try 0.85 or 0.95), and adds them as if they were labels. Three rounds, then the final model.

```
# Semi-supervised: pseudo-labelling (the same idea as Lab 2)
X_now, y_now = X_lab.copy(), y_lab.copy() # starts with the 5% real labels
pool = X_unlab.copy() # URLs without a label
rf_semi = RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1)
CUTOFF = 0.9 # try 0.85 or 0.95 too

for rnd in [1, 2, 3]:
    rf_semi.fit(X_now, y_now)
    p = rf_semi.predict_proba(pool)[: , 1]
    sure = (p >= CUTOFF) | (1 - p >= CUTOFF) # sure phishing OR sure legit
    guess = pd.Series((p[sure] >= 0.5).astype(int), index=pool.index[sure])
    X_now = pd.concat([X_now, pool[sure]]) # add them as labels
    y_now = pd.concat([y_now, guess])
    pool = pool[~sure]
    print(f"round {rnd}: added {sure.sum()} guesses,",
          f"{len(pool)} still unlabelled")

rf_semi.fit(X_now, y_now) # final model: labels + guesses
print("semi-supervised:", report(rf_semi, X_val, y_val))
```

What you should see

How many guesses each round added, then the scores. Do not be surprised if it is NOT better than the forest with 5% labels: on this dataset a forest is already strong with few labels. That is a real result: say so in your report.

5.3 Self-supervised: learn from the URLs alone first

Step A9: A fill-in-the-blanks network, then 5% labels

we hide 20% of the (scaled) values and train a small network to give back the complete row, without labels. Its middle layer has 32 numbers per URL. A logistic regression then uses those 32 numbers and the 5% labels. The Pipeline makes the whole thing work like any other model.

```
# Self-supervised: learn from the URLs alone, no labels (like Lab 3)
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import FunctionTransformer

scaler = StandardScaler().fit(X_train) # uses no labels
Z = scaler.transform(X_train)

# The made-up task: hide 20% of the clues and ask the network to fill them in
rng = np.random.default_rng(42)
hidden_part = rng.random(Z.shape) < 0.2
Z_masked = np.where(hidden_part, 0.0, Z) # 0 = average value after scaling

ae = MLPRegressor(hidden_layer_sizes=(32,), max_iter=300, random_state=42)
ae.fit(Z_masked, Z) # input: with holes, target: complete
print("fill-in error:", round(ae.loss_, 3))

def encode(X):
    """The 32 numbers the network learned for each URL (its middle layer)."""
    h = scaler.transform(X) @ ae.coefs_[0] + ae.intercepts_[0]
    return np.maximum(h, 0) # same "relu" as inside the network
```

```
# Now use the 5% labels, but on the 32 learned numbers
self_sup = Pipeline([
    ("encode", FunctionTransformer(encode)),
    ("lr", LogisticRegression(max_iter=1000)),
])
self_sup.fit(X_lab, y_lab)
print("self-supervised:", report(self_sup, X_val, y_val))
```

What you should see

The fill-in error (it only tells you the game was learned) and the scores. The fair comparison is logistic regression with 5% labels (step A7).

5.4 Compare everything

Step A10: All seven detectors on the same test set

```
models = {
    "tree": tree,
    "logreg": logreg,
    "forest (100%)": rf,
    "forest (5%)": rf_few,
    "logreg (5%)": logreg_few,
    "semi-supervised": rf_semi,
    "self-supervised": self_sup,
}
results = pd.DataFrame({name: report(m, X_test, y_test)
                        for name, m in models.items()}).T
print(results.round(3))
```

What you should see

A table with 7 rows. Usually the forest with all labels is best. Copy it into your report.

Check yourself

Did semi-supervised beat forest (5%)? Did self-supervised beat logreg (5%)? These two comparisons tell you whether the unlabelled data helped.

6. Part B: Explain the detectors

Step B1: Read the glass boxes

```
# Glass box 1: the tree, printed as rules
print(export_text(tree, feature_names=list(X_train.columns)))
print("tree leaves:", tree.get_n_leaves())

# Glass box 2: one weight per feature. Positive = pushes toward phishing.
w = pd.Series(logreg.named_steps["lr"].coef_[0], index=X_train.columns)
top10 = w.abs().sort_values(ascending=False).index[:10]
print(w[top10].round(3))
print("logreg weights with |w| > 0.01:", int((w.abs() > 0.01).sum()))
```

What you should see

The tree as text (each “|---” line is a yes/no question, “class: 1” = phishing), and the 10 strongest logistic regression weights (positive = pushes toward phishing).

Check yourself

Write the tree's first question in plain English. Does it make sense to a security person?

Step B2: SHAP for the supervised forest (global)

SHAP gives values for both classes, so we keep class 1 (phishing) with `sv[:, :, 1]`.

```
import shap
```

```

explainer = shap.TreeExplainer(rf)
sv = explainer(X_test.iloc[:500])      # 500 test URLs is plenty
print(sv.values.shape)                 # (500, 87, 2) = URLs, features, classes
sv1 = sv[:, :, 1]                      # keep class 1 = phishing

shap.plots.bar(sv1, max_display=10)     # average push of each feature
shap.plots.beeswarm(sv1, max_display=10) # one dot per URL

shap_top = pd.Series(np.abs(sv1.values).mean(axis=0),
                    index=X_test.columns).sort_values(ascending=False)
print(shap_top.head(10).round(4))

```

What you should see

The shape (500, 87, 2), a bar plot, a beeswarm plot and the top-10 list.

Check yourself

In the beeswarm, are the red dots of the top feature on the right or the left? What does that mean in plain words?

Step B3: SHAP for the semi- and self-supervised models

The semi-supervised model is a forest, so TreeExplainer works. The self-supervised model is not, so we use the general shap.Explainer with 100 ordinary URLs as the background, on 200 URLs (it is slower).

```

# Semi-supervised model: also a forest, so TreeExplainer works
sv_semi = shap.TreeExplainer(rf_semi)(X_test.iloc[:500])[:, :, 1]

# Self-supervised model: not a tree, so we use the general explainer.
# It works for any model but is slower, so we explain only 200 URLs.
def self_prob(a):
    a = pd.DataFrame(a, columns=X_train.columns)
    return self_sup.predict_proba(a)[:, 1]

background = X_train.sample(100, random_state=42) # "ordinary" URLs
explainer_self = shap.Explainer(self_prob, background)
sv_self = explainer_self(X_test.iloc[:200])
print(sv_semi.values.shape, sv_self.values.shape) # (500, 87) (200, 87)
shap.plots.beeswarm(sv_self, max_display=10)

```

Step B4: Do the models look at the same clues?

```

def top10(sv):
    s = pd.Series(np.abs(sv.values).mean(axis=0), index=X_test.columns)
    return s.sort_values(ascending=False).index[:10]

tree_top = pd.Series(tree.feature_importances_, index=X_train.columns)

lists = pd.DataFrame({
    "tree":      tree_top.sort_values(ascending=False).index[:10],
    "logreg":    w.abs().sort_values(ascending=False).index[:10],
    "SHAP forest": top10(sv1),
    "SHAP semi":  top10(sv_semi),
    "SHAP self":  top10(sv_self),
})
print(lists.to_string())

```

Check yourself

Which features appear in almost every list? Does the self-supervised model look at different clues from the two forests?

Step B5: Pick three URLs to explain

The phishing URL the forest is most sure about, the legitimate URL it is most sure about, and its most confident mistake (mistakes teach the most).

```
prob = rf.predict_proba(X_test)[: , 1]      # P(phishing) for every test URL
pred = (prob >= 0.5).astype(int)
yt = y_test.values

# the surest correct phishing alert
caught = np.where((yt == 1) & (pred == 1))[0]
i_phish = caught[np.argmax(prob[caught])]

# the surest correct legitimate URL
ok_legit = np.where((yt == 0) & (pred == 0))[0]
i_legit = ok_legit[np.argmin(prob[ok_legit])]

# the most confident mistake (or the least sure URL if there is no mistake)
wrong = np.where(yt != pred)[0]
if len(wrong) > 0:
    i_wrong = wrong[np.argmax(np.abs(prob[wrong] - 0.5))]
else:
    i_wrong = np.argmin(np.abs(prob - 0.5))

three = [("phishing", i_phish), ("legitimate", i_legit), ("wrong", i_wrong)]
for name, i in three:
    print(f"{name:10s} row {i}: P(phishing) = {prob[i]:.3f}",
          f"true label = {yt[i]}")
```

Step B6: Local explanations with SHAP and LIME

```
for name, i in three:
    one = explainer(X_test.iloc[[i]])[: , :, 1]  # this URL, class phishing
    print(name)
    shap.plots.waterfall(one[0], max_display=10)
```

LIME sends plain numbers to the model, so predict_fn puts the column names back.

```
from lime.lime_tabular import LimeTabularExplainer

def predict_fn(a, model=rf):
    # LIME sends plain numbers; give the model the column names it expects
    return model.predict_proba(pd.DataFrame(a, columns=X_train.columns))

lime_explainer = LimeTabularExplainer(
    training_data=X_train.values,
    feature_names=list(X_train.columns),
    class_names=["legitimate", "phishing"],
    mode="classification",
    discretize_continuous=True,      # LIME's default: features become ranges
    random_state=42)

for name, i in three:
    exp = lime_explainer.explain_instance(X_test.iloc[i].values, predict_fn,
                                         num_features=8, num_samples=5000)
    print(name, "- fit score:", round(exp.score, 2))
    for feature, weight in exp.as_list():
        print(f"    {feature:28s} {weight:+.3f}")
    exp.as_pyplot_figure()
```

What you should see

Three waterfall plots and three LIME charts with their fit scores. Red/right (SHAP) and positive weights (LIME) push toward phishing.

Check yourself

For the wrong URL: which features fooled the forest? Do SHAP and LIME name the same top 3 features?

7. Part C: Test the detector and the explanation

Step C1: Remove the external features (ablation)

An ablation changes one thing and keeps the rest the same. Here we retrain the forest without the 7 external features: they need live lookups, which are slow and not always available, and a brand-new phishing site has no history yet.

```
EXTERNAL = ["whois_registered_domain", "domain_registration_length",
            "domain_age", "web_traffic", "dns_record", "google_index",
            "page_rank"]

X_train_ne = X_train.drop(columns=EXTERNAL)
X_test_ne = X_test.drop(columns=EXTERNAL)

rf_ne = RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1)
rf_ne.fit(X_train_ne, y_train)

print("with external  :", report(rf, X_test, y_test))
print("without external:", report(rf_ne, X_test_ne, y_test))

sv_ne = shap.TreeExplainer(rf_ne)(X_test_ne.iloc[:500])[:, :, 1]
top_ne = pd.Series(np.abs(sv_ne.values).mean(axis=0), index=X_test_ne.columns)
print(top_ne.sort_values(ascending=False).head(5).round(4))
```

What you should see

The scores with and without the external features, and the new top 5 (now only URL and page features).

Check yourself

How much did macro-F1 drop? Would you deploy the model without live lookups? Why?

Step C2: Stability of LIME

Run LIME 5 times on the same URL with 5 different seeds and count how often the top 3 stays the same.

```
top3_sets = []
for seed in [0, 1, 2, 3, 4]:
    ex = LimeTabularExplainer(X_train.values, random_state=seed,
                             feature_names=list(X_train.columns))
    e = ex.explain_instance(X_test.iloc[i_phish].values, predict_fn,
                           num_features=8, num_samples=5000)
    top3 = frozenset(feature for feature, weight in e.as_list()[:3])
    top3_sets.append(top3)
    print("seed", seed, sorted(top3))

most_common = max(set(top3_sets), key=top3_sets.count)
print(top3_sets.count(most_common), "of 5 runs give the same top 3")
```

What you should see

How many of the 5 runs agree. SHAP TreeExplainer always gives the same answer (5 of 5). Write both in your report.

8. Part D: From explanations to rules (BRB)

Now you take the two most important features and build a BRB with them, the same way as in the BRB slides (Section 3.6).

Step D1: Choose your two features

We take the top of the forest's SHAP ranking (step B2), skipping 0/1 features like `google_index`, because a BRB needs Low, Medium and High. You may choose others by hand, e.g. `F1`, `F2` = "`page_rank`", "`nb_www`": say why in your report.

```
# The BRB needs Low / Medium / High, so skip 0/1 features
ranked = [f for f in shap_top.index if X_train[f].nunique() > 2]
F1, F2 = ranked[0], ranked[1]          # you may change these
print("Antecedents:", F1, "and", F2)
```

Step D2: The BRB code (example)

Step 1 splits a value between its two nearest reference values. Step 2 finds how strongly each of the 9 rules matches. Step 3 combines the rules with real Evidential Reasoning, so the output also has an Unknown part.

```
# Step 1: input transformation (same as in the BRB slides)
def transform_to_belief(value, references):
    value = float(value)
    n = len(references)
    belief = [0.0] * n
    if value <= references[0]:
        belief[0] = 1.0
        return belief
    if value >= references[-1]:
        belief[-1] = 1.0
        return belief
    for i in range(n - 1):
        lower, upper = references[i], references[i + 1]
        if lower <= value <= upper:
            d = upper - lower
            belief[i] = (upper - value) / d
            belief[i + 1] = (value - lower) / d
            return belief
    return belief

# Step 2: rule activation weights (9 rules = 3 x 3)
def calculate_activation_weights(belief1, belief2):
    d1 = attribute_weights[0]
    d2 = attribute_weights[1]
    activation = []
    for i in range(3):
        for j in range(3):
            w = (rule_weights[i * 3 + j]
                  * belief1[i] ** d1 * belief2[j] ** d2)
            activation.append(w)
    total = sum(activation)
    if total == 0:
        return [0.0] * len(activation)
    return [x / total for x in activation]

# Step 3: Evidential Reasoning (ER)
def combine_two(b1, ig1, b2, ig2, n=3):
    conflict = sum(b1[i] * b2[j] for i in range(n)
                   for j in range(n) if i != j)
    k = 1 / (1 - conflict)
    new_b = [k * (b1[i] * b2[i] + b1[i] * ig2 + ig1 * b2[i])
              for i in range(n)]
    return new_b, k * (ig1 * ig2)

def er_aggregation(weights, rules, n=3):
    act = [(w, r) for w, r in zip(weights, rules) if w > 0]
    bel = [[w * v for v in r] for w, r in act]
    ign = [1 - sum(b) for b in bel]
    fb, fg = bel[0], ign[0]
```

```

    for i in range(1, len-bel)):
        fb, fg = combine_two(fb, fg, bel[i], ign[i], n)
    return fb, fg

def brbes_inference(v1, v2):
    belief1 = transform_to_belief(v1, refs1)
    belief2 = transform_to_belief(v2, refs2)
    aw = calculate_activation_weights(belief1, belief2)
    final_belief, ignorance = er_aggregation(aw, rules)
    return belief1, belief2, aw, final_belief, ignorance

```

Step D3: Referential values and the 9 rules

Low / Medium / High are the 5th percentile, median and 95th percentile of the training data. In the slides an expert wrote the rules by hand; here we start from the data: for each rule we count how many of the training URLs that match it are phishing (share 0 = Low risk, 0.5 = Medium, 1 = High). The **support** says how much data stands behind each rule.

```

# Referential values Low / Medium / High for each antecedent,
# taken from the TRAINING data: 5th percentile, median, 95th percentile
refs1 = X_train[F1].quantile([0.05, 0.50, 0.95]).round(2).tolist()
refs2 = X_train[F2].quantile([0.05, 0.50, 0.95]).round(2).tolist()
print(F1, "Low/Medium/High =", refs1)
print(F2, "Low/Medium/High =", refs2)

consequent_refs = ["Low", "Medium", "High"] # consequent: phishing risk
attribute_weights = [1.0, 1.0]
rule_weights = [1.0] * 9

# Start the 9 rules from the training data.
# For each rule: how many of the URLs that activate it are phishing?
A = np.array([calculate_activation_weights(transform_to_belief(a, refs1),
                                         transform_to_belief(b, refs2))
              for a, b in X_train[[F1, F2]].values]) # (6858 URLs, 9 rules)
support = A.sum(axis=0) # how much data per rule
phish = (A * y_train.values[:, None]).sum(axis=0) # phishing part
share = phish / np.maximum(support, 1e-9)

# Turn each phishing share into beliefs in Low / Medium / High risk
# (share 0 -> Low, 0.5 -> Medium, 1 -> High), with the same Step 1 function
rules = [transform_to_belief(s, [0.0, 0.5, 1.0]) for s in share]

names = ["Low", "Med", "High"]
for k in range(9):
    i, j = k // 3, k % 3
    print(f"R{k + 1}: {F1}={names[i]:4s} {F2}={names[j]:4s} "
          f"-> {[round(x, 2) for x in rules[k]]} (support {support[k]:.0f})")

```

Check yourself

Do the rules make sense? Act as the expert: change one rule by hand (e.g. `rules[3] = [0.0, 0.2, 0.8]`), rerun steps D4 and D5, and see what changes. Rerun step D3 to put it back.

Step D4: Use the BRB as a phishing detector

BRBDetector turns the beliefs into one phishing score with the utility, so report works unchanged. For a fair comparison we also train a small tree and a forest on the same two features.

```

def brb_predict_proba(X):
    """X: array (n, 2) of [F1, F2].
    Returns (n, 4): Low, Medium, High, Unknown."""
    X = np.atleast_2d(X)
    out = []
    for v1, v2 in X:
        fb, ig = brbes_inference(v1, v2)[3:]

```

```

        out.append(fb + [ig])
    return np.array(out)

class BRBDetector:
    """Makes the BRB look like a scikit-learn model, so report() works."""
    def predict_proba(self, X):
        B = brb_predict_proba(X[[F1, F2]].values)
        # utility: Low = 0, Medium = 0.5, High = 1; Unknown is left out
        u = (0.5 * B[:, 1] + B[:, 2]) / (1 - B[:, 3])
        return np.column_stack([1 - u, u])
    def predict(self, X):
        return (self.predict_proba(X)[:, 1] >= 0.5).astype(int)

brb = BRBDetector()

# Fair comparison: other models that see ONLY the same two features
tree2 = DecisionTreeClassifier(max_depth=2, random_state=42)
tree2.fit(X_train[[F1, F2]], y_train)
rf2 = RandomForestClassifier(n_estimators=300, random_state=42,
                             n_jobs=-1)
rf2.fit(X_train[[F1, F2]], y_train)

print("BRB, 2 features      :", report(brb, X_test, y_test))
print("tree, 2 features     :", report(tree2, X_test[[F1, F2]], y_test))
print("forest, 2 features   :", report(rf2, X_test[[F1, F2]], y_test))
print("forest, all 87      :", report(rf, X_test, y_test))

```

Check yourself

The BRB and the small tree have almost the same macro-F1. Look at recall and FAR: which would a security team prefer?

Step D5: Explain one URL: the rules, LIME and SHAP

First the BRB explains itself: which rules fire for the forest's mistake from step B5, and how much is Unknown. Then we treat the BRB as a black box, as in the slides, and explain its High-risk belief with LIME and SHAP.

```

# The BRB explains itself: which rules fire for the mistake from step B5?
url = X_test[[F1, F2]].iloc[i_wrong].values
b1, b2, aw, fb, ig = brbes_inference(*url)
print(F1, "=", url[0], " ", F2, "=", url[1])
for k in range(9):
    if aw[k] > 0:
        print(f"    rule R{k + 1} fires with weight {aw[k]:.2f}")
print("belief Low/Medium/High:", [round(x, 3) for x in fb])
print("Unknown:", round(ig, 3), " true label:", y_test.iloc[i_wrong])

```

```

feature_names = [F1, F2]
X_ref = X_train[[F1, F2]].values          # reference data for LIME and SHAP

brb_lime = LimeTabularExplainer(X_ref, feature_names=feature_names,
                                discretize_continuous=False,
                                random_state=42)
lime_exp = brb_lime.explain_instance(url, brb_predict_proba,
                                     labels=(2,),          # 2 = "High" risk
                                     num_features=2, num_samples=2000)
print("LIME (High):", lime_exp.as_list(label=2))
print("fit score   :", round(lime_exp.score, 3))

```

```
background = shap.sample(X_ref, 100, random_state=0) # "ordinary" URLs
brb_shap = shap.KernelExplainer(brb_predict_proba, background)
sv_brb = brb_shap(url.reshape(1, -1))
sv_brb.feature_names = feature_names
shap.plots.waterfall(sv_brb[0, :, 2]) # the "High" risk belief
```

What you should see

The rules that fire and the Unknown part, LIME's two weights with its fit score, and a SHAP waterfall plot for High risk.

Check yourself

Was the BRB fooled too, and which rule is most to blame? Do LIME and SHAP agree on which input pushes High risk most?

9. What to hand in

Your code: one notebook that runs from top to bottom, plus a short README (libraries, dataset, how to run). Keep `random_state=42`. **A short report (2–3 pages)** with these headings:

Heading	What to write
1. Problem	One paragraph: the phishing task, the three ways of learning, glass box vs post-hoc.
2. Setup	Dataset, splits, label budget (5%), models and explainers (a short table is fine).
3. Results	The step A10 table; one SHAP and one LIME figure with captions; the top-10 lists (B4); the ablation (C1) and LIME stability (C2); your two BRB features, the 9 rules and the BRB scores (Part D).
4. Discussion	Did the unlabelled data help? Do the models look at the same clues? Which model would you deploy, and why? What does the mistake show? Which top features can an attacker fake? Do LIME and SHAP agree with your BRB rules? Answer the “Check yourself” questions here.
5. Who did what	One or two lines.

How your work is graded

Every lab is graded 3 / 4 / 5 / F. You do not need to be perfect to pass: the work must run, make sense, and be clearly reported.

Criterion	Weight	What good work looks like here
Implementation and code quality	30%	The notebook runs start-to-finish and reproduces your numbers; all three ways of learning and the BRB work; README; seed is fixed.
Experimental design and methodology	25%	Stratified splits; the 5% split made from the training set only and <code>y_hidden</code> never used for training; both lower lines; all models compared on the same test set; the ablation and the LIME stability test; BRB reference values from the training data only.
Results and analysis	20%	Complete tables; honest verdict on whether the unlabelled data helped; explanations compared; the mistake explained; attacker view; BRB rules compared with LIME and SHAP.
Report quality	25%	Clear 2–3 pages; captioned figures and tables used in the text; who-did-what line included + screenshot of your code.

Submitting your work

- **Where.** Upload to CANVAS by the deadline: your code (a zip or a repository link) and the report as a PDF. Late submissions are not accepted in general.
- **Group work.** Add a short line saying who did what. If one person does everything, that shows up in individual grades.
- **Second chance.** If a lab does not reach grade 3 you may resubmit it once, before the final reporting date.
- **Honesty.** You may use libraries, tutorials and public code, just cite them. If you use an AI assistant, say so in the report.